BCSE305L
EMBEDDED SYSTEMS

Dr.N.Anusha

COURSE OUTCOMES

1. Identify the challenges in designing an embedded system using various microcontrollers and interfaces.
2. To summaries the functionality of any special purpose computing system, and to propose smart solutions to engineering challenges at the prototype level.
3. To examine the working principle and interface of typical embedded system components, create programme models, apply various optimization approaches including simulation environment and demonstration using debugging tools.
4. To evaluate the working principle of serial communication protocols and their proper use, as well as to analyze the benefits and drawbacks of real-time scheduling algorithms and to recommend acceptable solutions for specific challenges

Module 1 --- Introduction

- Overview of Embedded Systems
- Design challenges
- Embedded processor technology
- Hardware Design
- Micro-controller architecture -8051, PIC, and ARM

Definition of Embedded Systems

- Embedded system is designed for a specific application

- Examples of ES ???

Examples of Embedded Systems

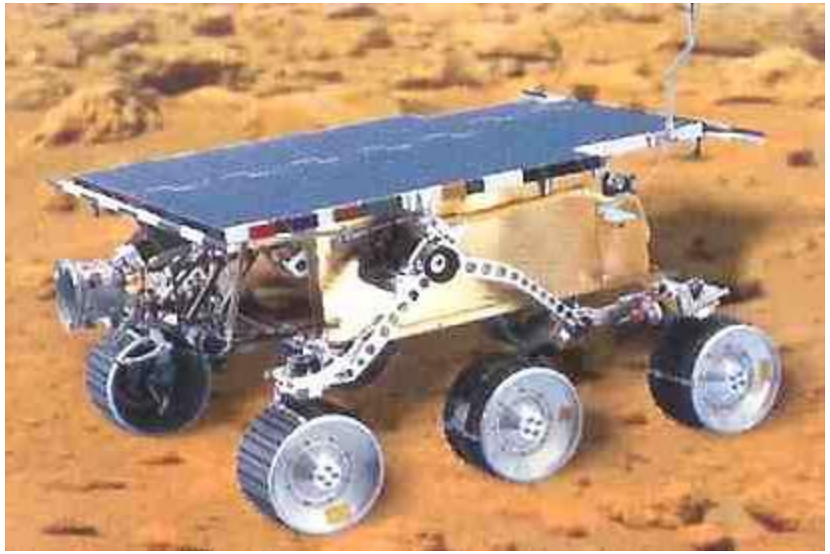
Many Different Products Depend on Embedded Systems

Examples of Embedded Systems

More Examples

- Miele dishwashers uses 8b Motorola 68HC05 processors and can be reprogrammed with a laptop to adjust temperatures and cycle times

More Examples



- NASA Mars Rover uses an Intel 80C85 8-bit microprocessor

More Examples

- Palm Vx Handheld uses Motorolla Dragonball EZ 32-b microprocessor

More Examples

- Philips sonicare plus toothbrush – Zilog Z8 8b microprocessor

More Examples

- Vendo V-MAX 720 vending machine uses the Motorola M68HC11 8b microprocessor
  - electronic and mechanical parts should go in hand
  - delivering a good wrt cash - transformed - web enabled cashless delivery
    - stock monitored remotely
    - transactions through credit or smart card
    - security monitor from remote

More Examples

- Sony's Aibo Robotic Dog uses ERS-110 an MIPS 64b RISC processor
  - coordinate the motions
  - needs to do sensing
  - control the manipulators
  - need to communicate
    - Ex: football competition b/w robo

More Examples

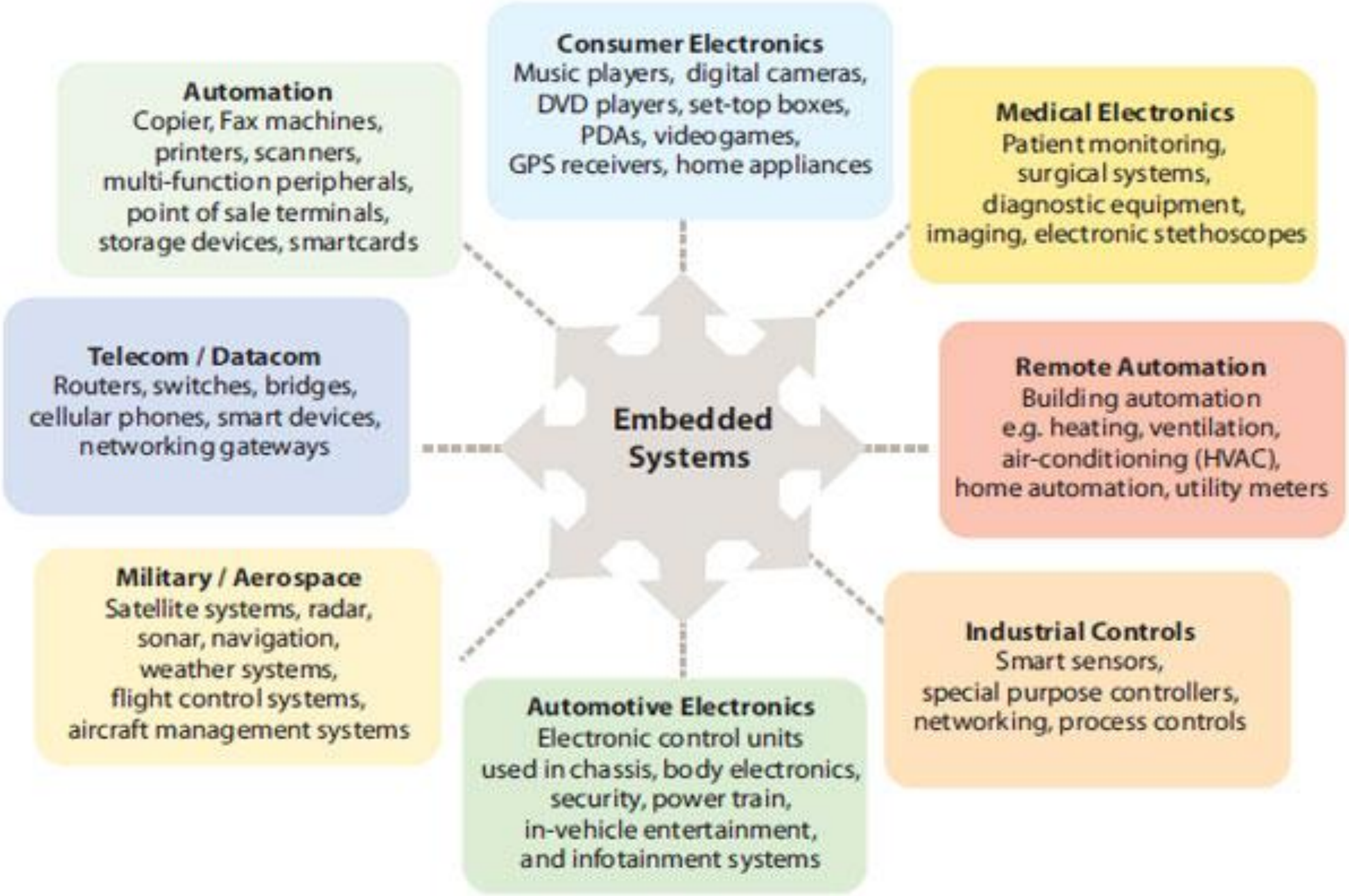
- Rio MP3 Player uses 32b RISC processor
  - compressed form of audio is mp3
  - sophisticated up to be used for audio processing

More Examples

- Garmin Streetpiolot GPS receiver uses a 16b processor
  - any transport system can get its global location
  - automated navigation system make use GPS
  - communication with satellite and output about vehicle position and map display

More Examples

- Hunter 44550 programmable thermostat uses a 4b processor


Consumer Electronics
Music players, digital cameras,
DVD players, set-top boxes,
PDAs, videogames,
GPS receivers, home appliances
Automation
Copier, Fax machines,
printers, scanners,
multi-function peripherals,
point of sale terminals,
storage devices, smartcards
Medical Electronics
Patient monitoring,
surgical systems,
diagnostic equipment,
imaging, electronic stethoscopes
Telecom / Datacom
Routers, switches, bridges,
cellular phones, smart devices,
networking gateways
Embedded Systems
Remote Automation
Building automation
e.g. heating, ventilation,
air-conditioning (HVAC),
home automation, utility meters
Military / Aerospace
Satellite systems, radar,
sonar, navigation,
weather systems,
flight control systems,
aircraft management systems
Industrial Controls
Smart sensors,
special purpose controllers,
networking, process controls
Automotive Electronics
Electronic control units
used in chassis, body electronics,
security, power train,
in-vehicle entertainment,
and infotainment systems

Regular VS Embedded Software Development

Regular Software Development:

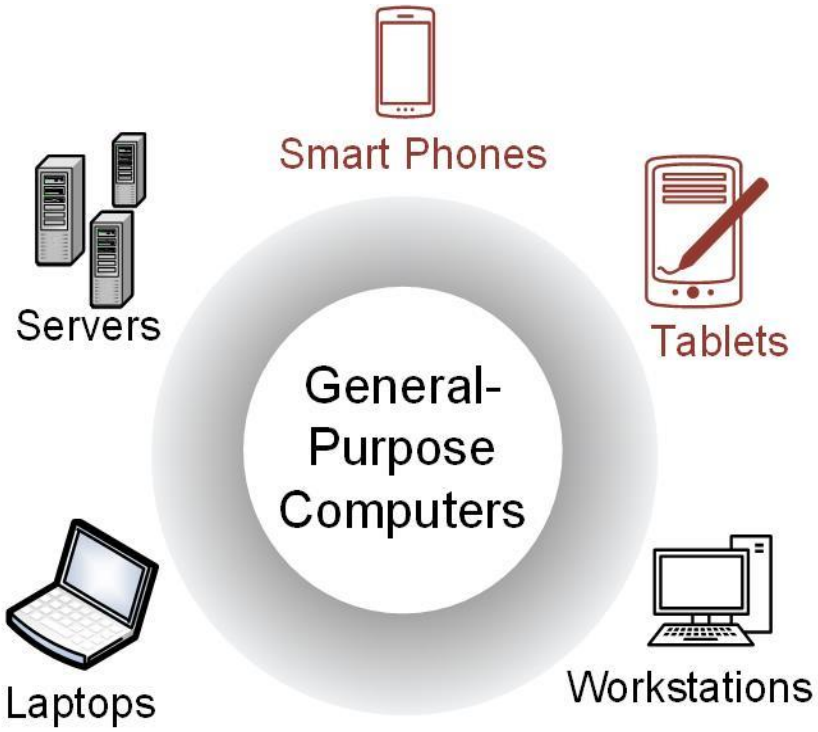
- Client-Server Architecture
- Operating System Compatibility
- Platform Independent
- User Interface Design
- Rapid Application Development
- Standardized Development Frameworks
- Scalable Architecture
- Widely Used Technologies
- Web Development Technologies
- Wide Range of Development Tools

Embedded Software Development:

- Specific Hardware Platform
- Real-Time Constraints
- Firmware Development
- Memory Management
- Efficient Power Consumption
- Low-Level Programming Languages
- Hardware-Software Integration
- Performance Optimization
- Customized Development Process
- Minimalistic User Interface

GENERAL-PURPOSE COMPUTERS

- Able to run a variety of software.
- Contain relatively high-performance hardware components (fast processors, data & program storage).
- Require an operating system (OS).

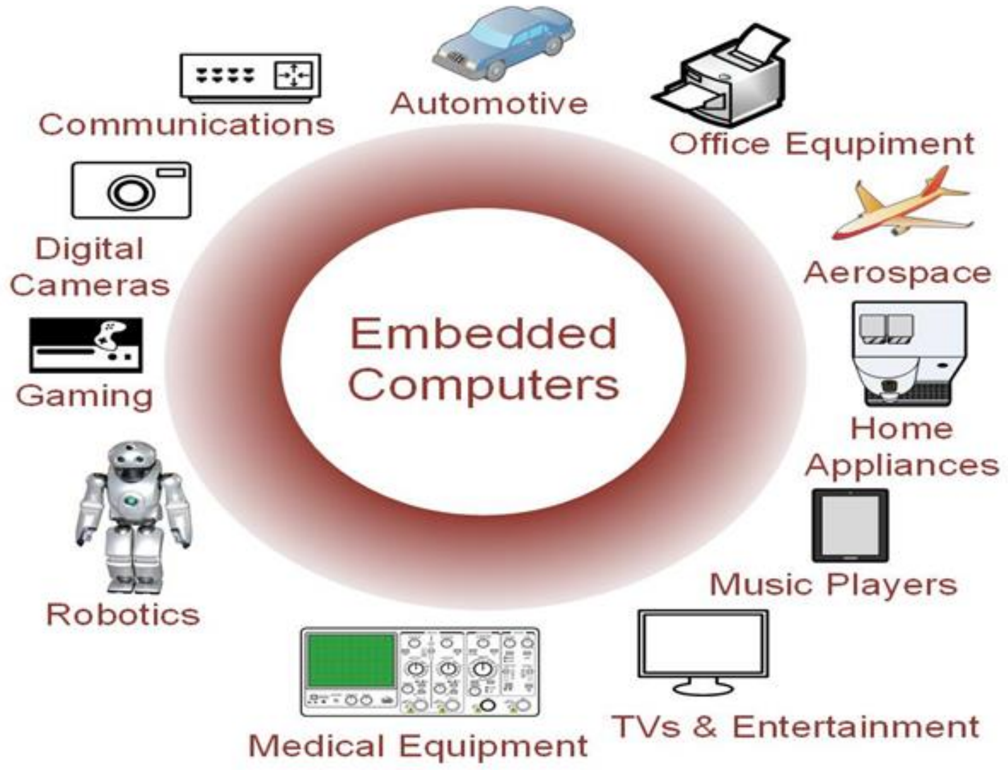


GENERAL-PURPOSE COMPUTERS

- Designed for heavy user interaction.
- Uses a variety of peripherals (displays, keyboards, mice, internet connections, wireless communication capability).
- Expensive ($100s - $1000s).
- Use a group of integrated circuits or chips (ICs).
  - One implements the central processing unit (CPU).
- Several implement data memory and program storage.

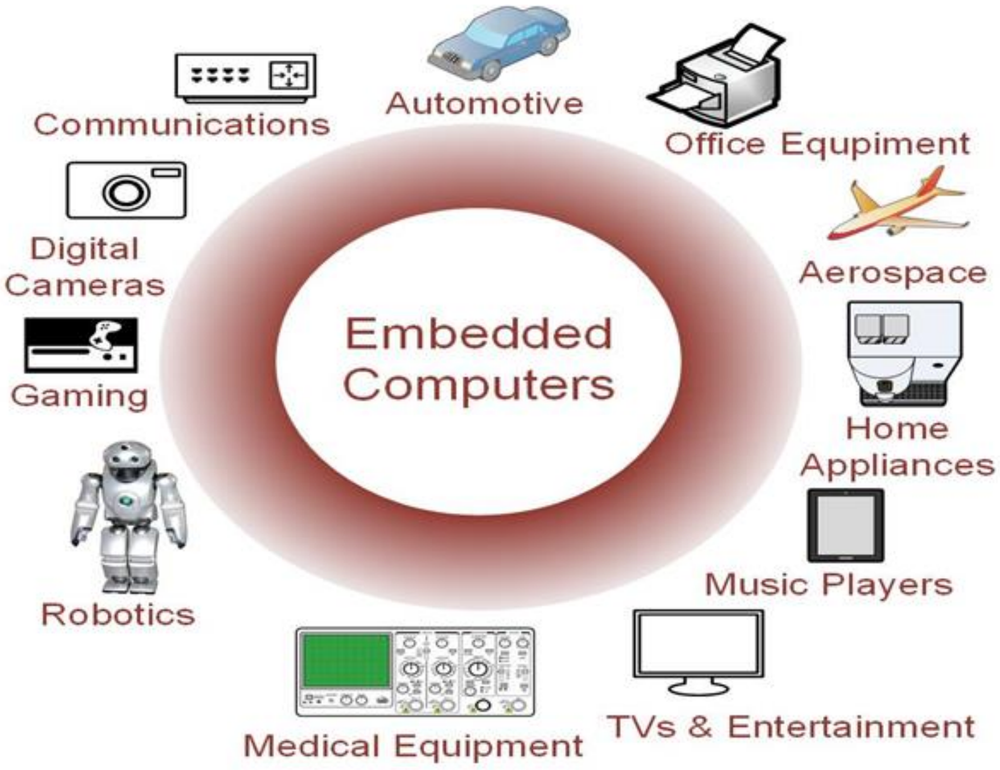
EMBEDDED COMPUTERS

- Resources can be implemented on a single IC.
- Include a variety of peripherals (timers, analog-to-digital converters, digital-to-analog converters, serial interfaces).
- Small size makes them very versatile.

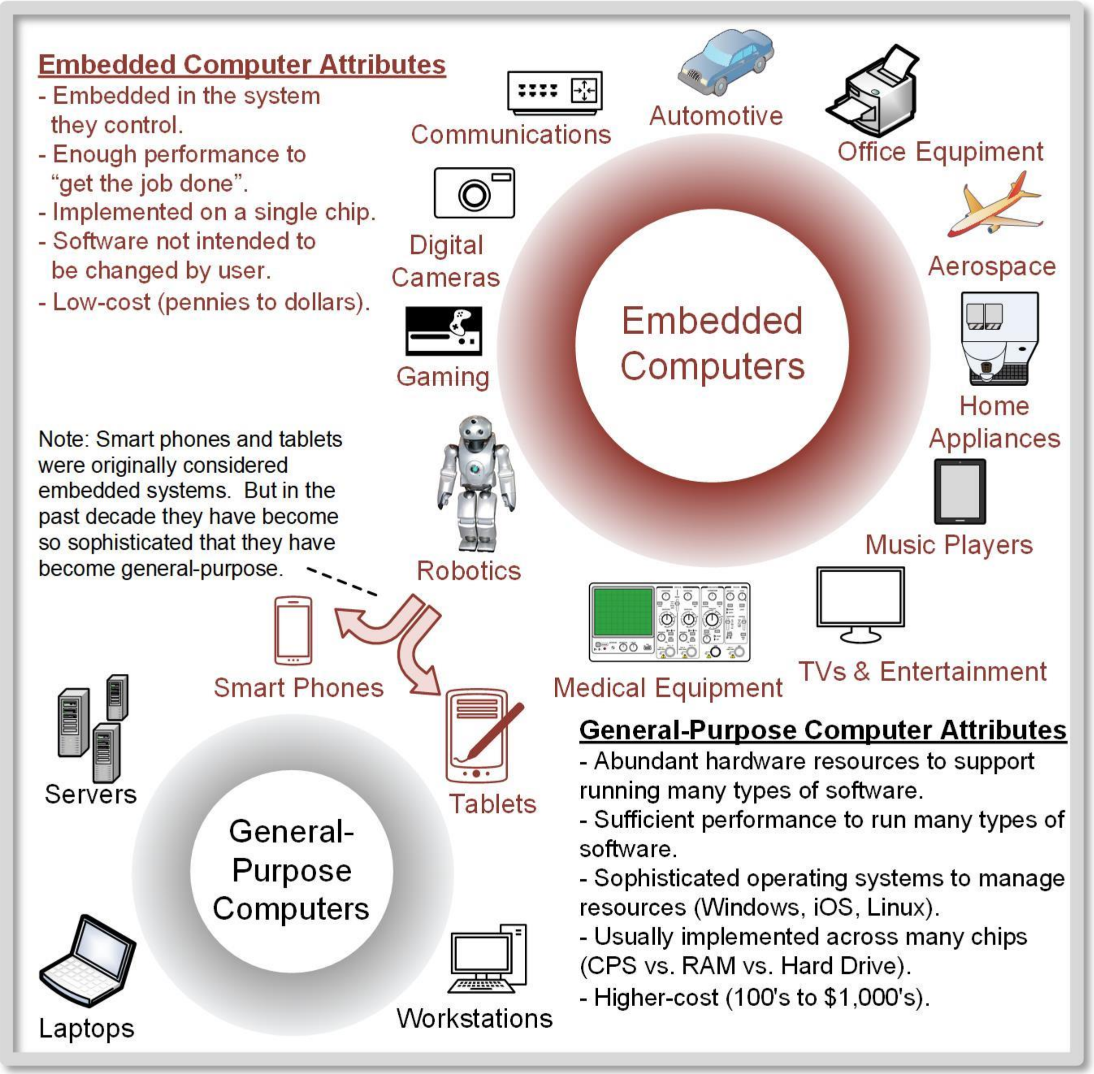


Embedded Computers

- Contains firmware (only the *needed* software which is not intended to be changed frequently).
- May contain Real Time Operating Systems (RTOS) which are used as a task scheduler.
- Low cost (10s of cents to a few dollars).



Embedded Computers



What is a System?

- System Definition

- A way of working, organizing or performing one or many tasks according to a fixed set of rules, program or plan.

- Example
  - Time display system – A watch
  - Automatic cloth washing system – A washing machine

What is Embedded System?

- Embedded System
  - any device that includes a computer but is not itself a general purpose computer
  - h/w and s/w - part of some larger systems and expected to function without human intervention
  - respond, monitor, control external environment using sensors and actuators
- embedding a computer - but not for general purpose
- Applied Computer System
- Includes analog interface to the external world

Embedded System - Definitions

- An embedded system is an application that contains at least one programmable computer (typically in the form of a microcontroller, a microprocessor or digital signal processor chip) is used by individuals who are, in the main, unaware that the system is computer-based
    - From Embedded C Programming perspective by Michael.J.Pont

- "An embedded system is a system that has software embedded into computer-hardware, which makes a system dedicated for an application (s) or specific part of an application or product or part of a larger system."
  - s/w usually embeds into a ROM or flash
  - Independent system or part of a large system
    - by Raj Kamal

Embedded System - Definitions

- “An embedded system is one that has a dedicated purpose software embedded in a computer hardware.”
    - By Raj Kamal

- Any device that includes programmable computer but not self intend to be a general purpose computer
    - Wayne Wolf

- Electronic system contains microcontroller or microprocessor but not general purpose computers computer is hidden or embedded in the system
    - Todd D. Morton

Embedded System - Definitions

- Combination of Software and Hardware in which the software controls the entire hardware for a dedicated application

  -Raj Kamal

- A general-purpose definition of embedded systems is that they are devices used to control, monitor or assist the operation of equipment, machinery or plant. "Embedded" reflects the fact that they are an integral part of the system. In many cases, their "embeddedness" may be such that their presence is far from obvious to the casual observer.

  Institute of Electrical Engineers (IEE)

Major components of Embedded system

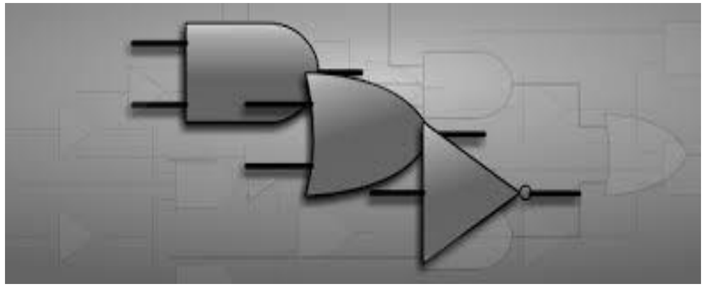
- Hardware
- Software
- RTOS

Embedded System Vs Desktop System

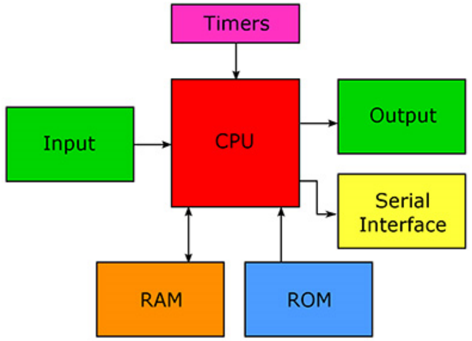
- Desktop / Laptop
  - General purpose computer
  - Used for playing games, word processing, accounting, SDT etc.,

- Embedded System
  - Single Purpose and
  - fixed embedded software for specific job

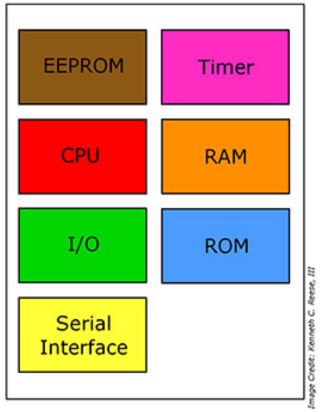
- Typical Examples
  - A/C, VCD/DVD Player, Printer, Fax m/c, Mobile phone etc
  - Customized embedded hw + fixed embedded sw (firmware) + specific processor
    - to meet the specific requirement

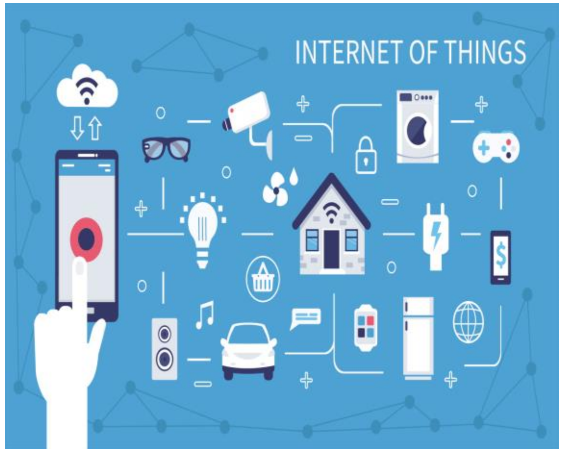


<u>Microprocesser</u>: CPU
and several supporting chips.



<u>Microcontroller</u>: CPU
on a single chip.





Soft Real-Time Systems
vs
Hard Real-Time Systems

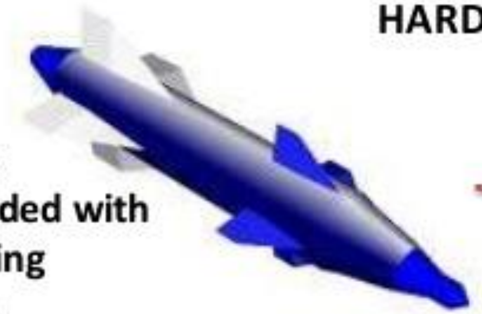
- **Soft real-time** is when a system continues to function even if it's unable to execute within an allotted time. If the system has missed its deadline, it **will not result in critical consequences**. The system can continue to function, though with undesirable lower quality of output.

- **Hard real-time** is when a system will cease to function if a deadline is missed, which **can result in catastrophic consequences.**

Real Time Embedded Systems

A system in which work has to be done in a specific time period.

HARD REAL TIME

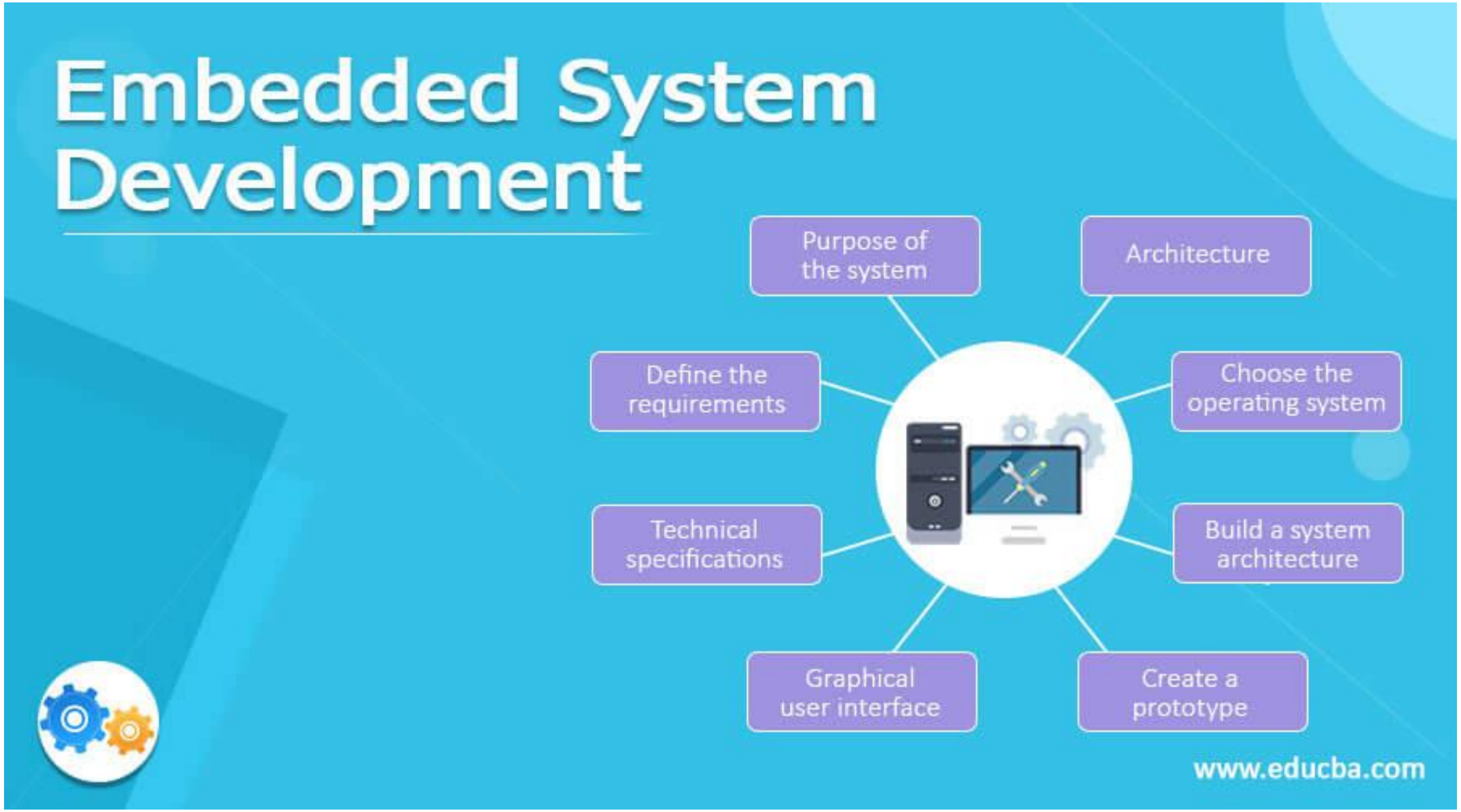


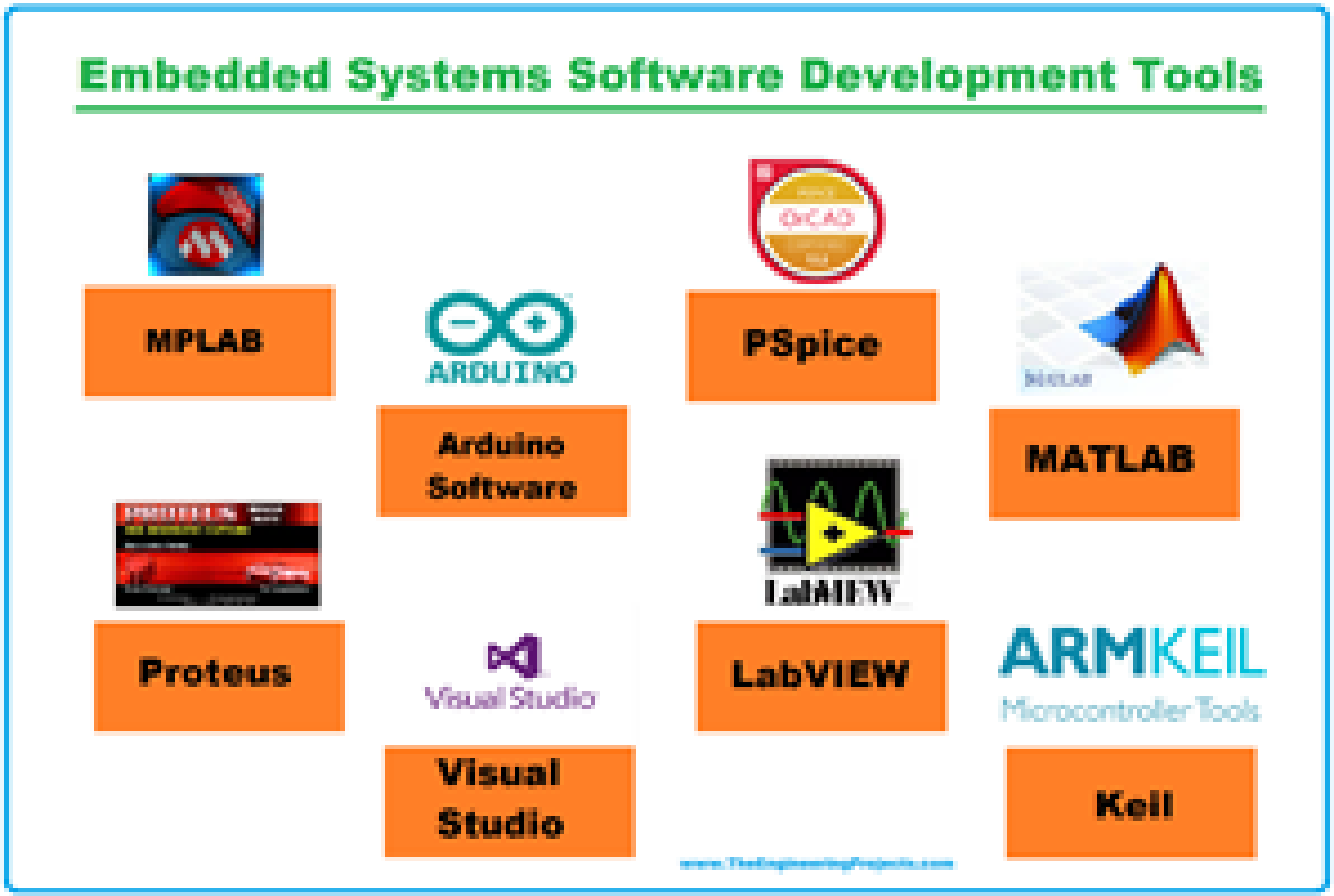
Missile embedded with a tracking system

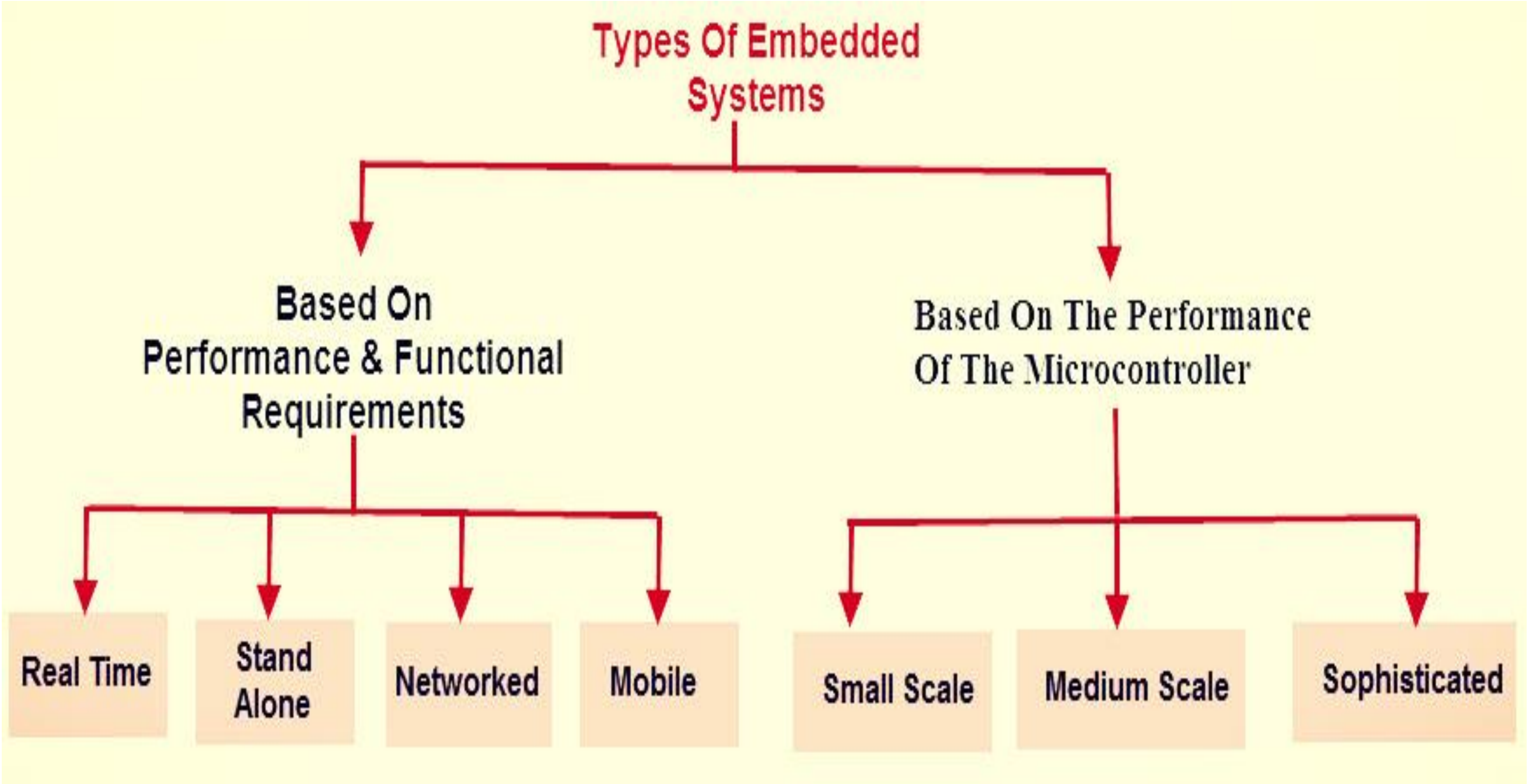
Aircraft

SOFT REAL TIME

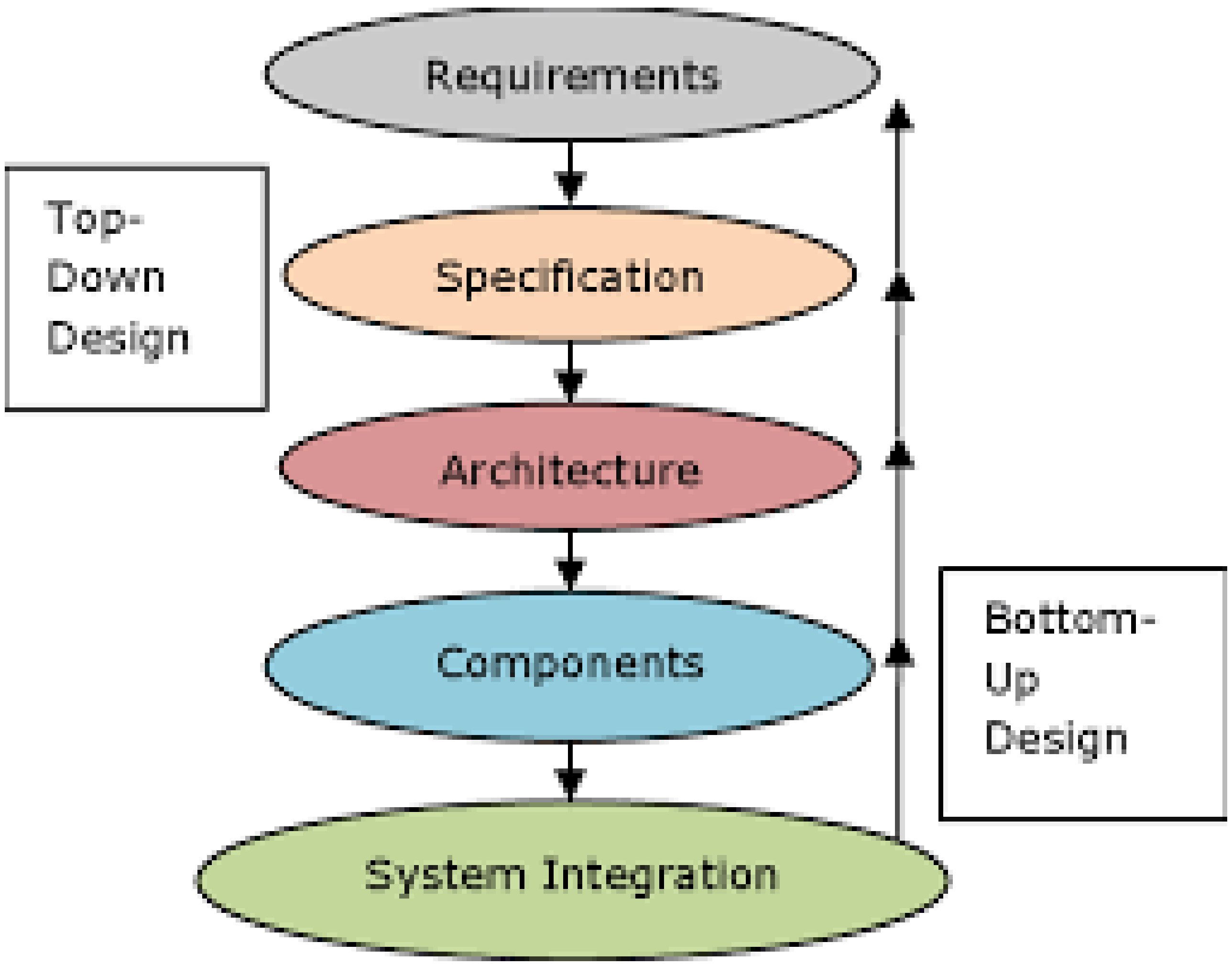
DVD PLAYER


Embedded System Development
Purpose of the system
Architecture
Define the requirements
Choose the operating system
Technical specifications
Build a system architecture
Graphical user interface
Create a prototype
www.educba.com


Embedded Systems Software Development Tools
MPLAB
ARDUINO
Arduino Software
PSpice
MATLAB
Proteus
Visual Studio
Visual Studio
LabVIEW
LabVIEW
ARMKEIL
Microcontroller Tools
Keil


Types Of Embedded Systems
Based On Performance & Functional Requirements
Based On The Performance Of The Microcontroller
Real Time
Stand Alone
Networked
Mobile
Small Scale
Medium Scale
Sophisticated

Embedded System Design Process



Top-down vs. bottom-up

- Top-down design:
  - start from most abstract description;
  - work to most detailed.
- Bottom-up design:
  - work from small components to big system.
- Real design uses both techniques.

Stepwise refinement

- At each level of abstraction, we must:
  - analyze the design to determine characteristics of the current state of the design;
  - refine the design to add detail.

Requirements

- Plain language description of what the user wants and expects to get.
- May be developed in several ways:
  - talking directly to customers;
  - talking to marketing representatives;
  - providing prototypes to users for comment.

Functional vs. non-functional requirements

- Functional requirements:
  - output as a function of input.
- Non-functional requirements:
  - time required to compute output;
  - size, weight, etc.;
  - power consumption;
  - reliability;
  - etc.

Requirements form

- name
- purpose
- inputs
- outputs
- functions
- performance
- manufacturing cost
- power
- physical size/weight

GPS moving map requirements form

name	GPS moving map
purpose	consumer-grade moving map for driving
inputs	power button, two control buttons
outputs	back-lit LCD 400 X 600
functions	5-receiver GPS; three resolutions; displays current lat/lon
performance	updates screen within 0.25 sec of movement
manufacturing cost	$100 cost-of-goods-sold
power	100 mW
physical size/weight	no more than 2: X 6:, 12 oz.

Specification

- A more precise description of the system:
  - should not imply a particular architecture;
  - provides input to the architecture design process.
- May include functional and non-functional elements.
- May be executable or may be in mathematical form for proofs.

GPS specification

- Should include:
  - What is received from GPS;
  - map data;
  - user interface;
  - operations required to satisfy user requests;
  - background operations needed to keep the system running.

Architecture design

- What major components go satisfying the specification?
- Hardware components:
  - CPUs, peripherals, etc.
- Software components:
  - major programs and their operations.
- Must take into account functional and non-functional specifications.

Designing hardware and software components

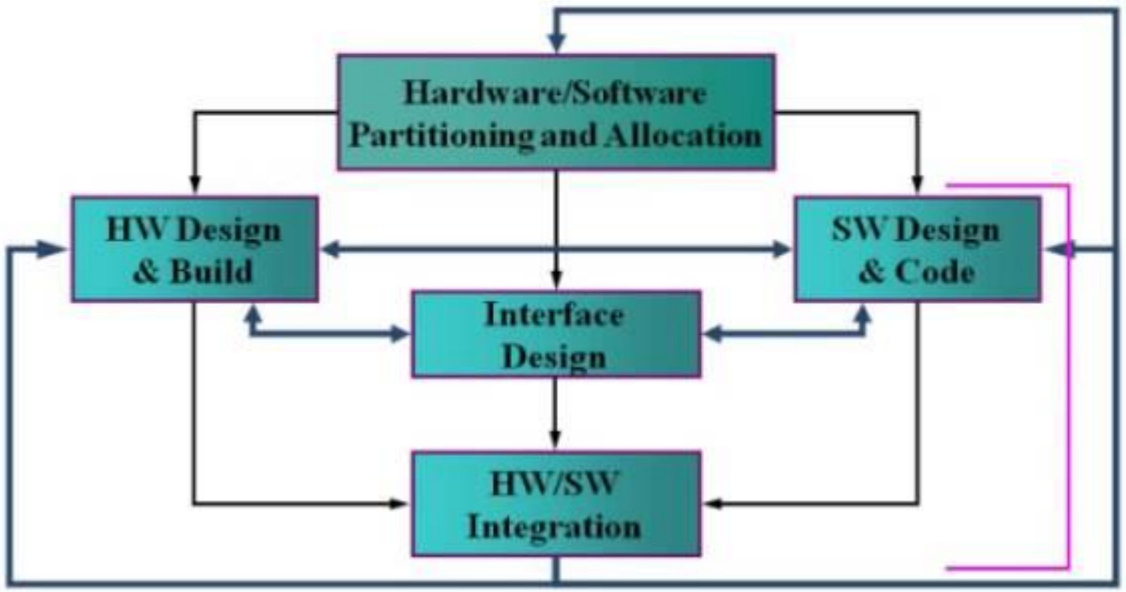
- Must spend time architecting the system before you start coding.
- Some components are ready-made, some can be modified from existing designs, others must be designed from scratch.

System integration

- Put together the components.
  - Many bugs appear only at this stage.
- Have a plan for integrating components to uncover bugs quickly, test as much functionality as early as possible.

Embedded System Design

HW/SW Co-Design Methodology



Characteristics of Embedded Systems

- Low power consumption
- Optimized system failure rate
- Little to no maintenance overhead and human involvement
- Increased resistance to external factors
- Minimized size, weight, and costs
- Sole task completion
- Continuous and uninterrupted operation
- Simplistic programming
- Exceptional levels of fault tolerance

TATEEDA

Characteristics of embedded systems- Marilyn wolf

- Sophisticated functionality.
- Real-time operation with time constraints
- Low manufacturing cost.
- Low power
- Designed to tight deadlines by small teams.

Characteristics of embedded systems-Frank G Vahid

- Single Functioned
- Time constraints
- Tightly Constraint
- Real Time & Reactive
- Complex Algorithms
- User Interface
- Multirate
- Manufacturing Cost
- Power

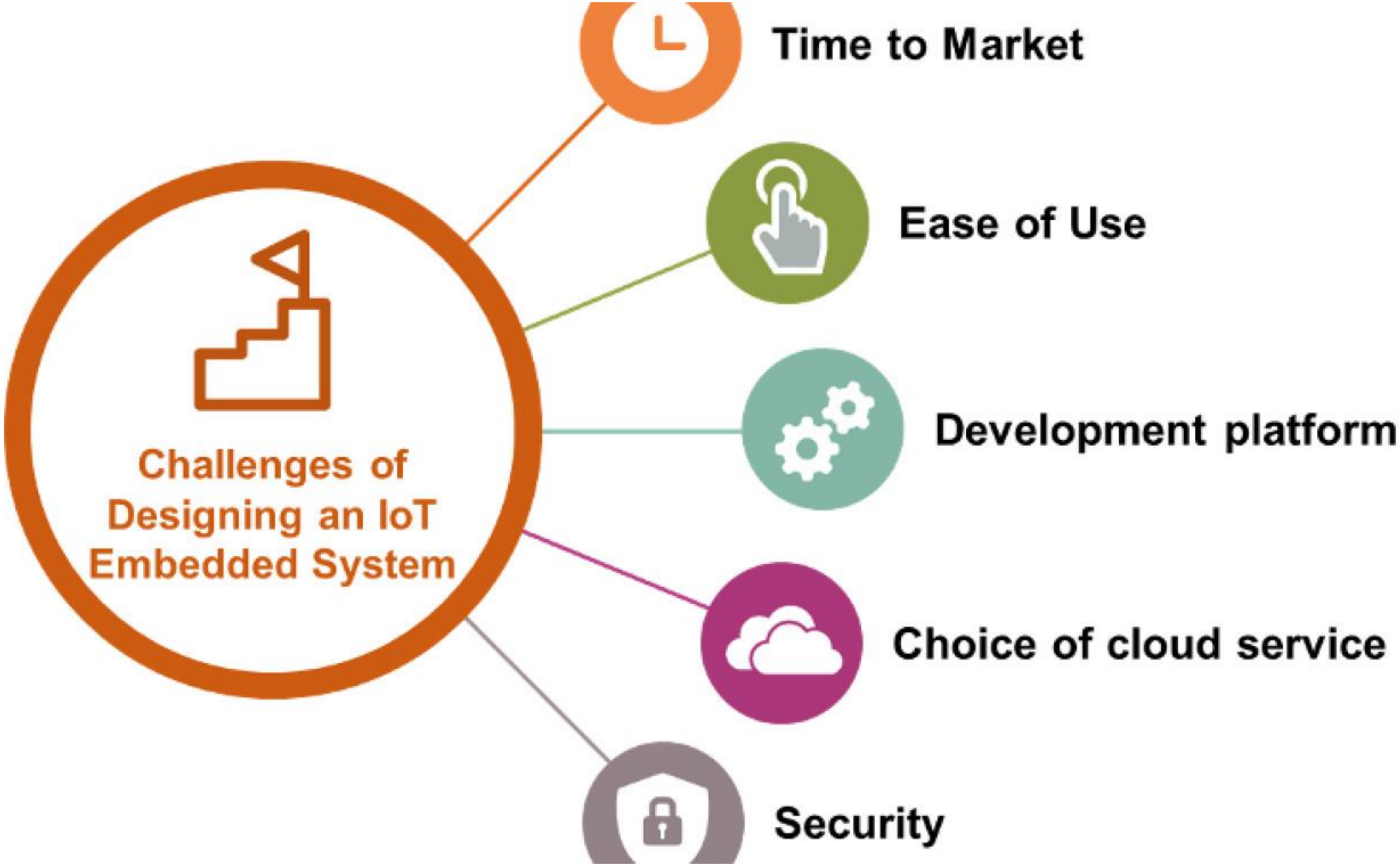
What does “performance” mean?

- In general-purpose computing, performance often means average-case, may not be well-defined.
- In real-time systems, performance means meeting deadlines.
  - Missing the deadline by even a little is bad.
  - Finishing ahead of the deadline may not help.

Characterizing performance

- We need to analyze the system at several levels of abstraction to understand performance:
  - CPU.
  - Platform.
  - Program.
  - Task.
  - Multiprocessor.

Challenges in Embedded system


Challenges of Designing an IoT Embedded System
Time to Market
Ease of Use
Development platform
Choice of cloud service
Security

Design challenge – optimizing design metrics

- Obvious design goal:
  - Construct an implementation with desired functionality
- Key design challenge:
  - Simultaneously optimize numerous design metrics
- Design metric
  - A measurable feature of a system's implementation
  - Optimizing design metrics is a key challenge

Design challenge – optimizing design metrics

- Common metrics
  - NRE cost (Non-Recurring Engineering cost):

    The one-time monetary cost of designing the system. Once the system is designed, any no of units can be manufactured without incurring any additional design cost.
  - Unit cost:
    - the monetary cost of manufacturing each copy of the system, excluding NRE cost
  - Size:
    - The physical space required by the system.
    - Software - Bytes
    - Hardware – gates and transistors
  - Performance:
    - The execution time or throughput of the system

Design challenge – optimizing design metrics

- Common metrics (continued)
  - Power: The amount of power consumed by the system
  - Flexibility:
    - The ability to change the functionality of the system without incurring heavy NRE cost
  - Time-to-prototype:
    - The time needed to build a working version of the system
  - Time-to-market:
    - The time required to develop a system to the point that it can be released and sold to customers
  - Maintainability:
    - The ability to modify the system after its initial release.
  - Correctness, safety

Why is Design of Embedded Systems Difficult?

- High Complexity
- Strong time and power constraints
- Low cost
- Short time to market
- Safety critical systems
- In order to achieve all these requirements, systems have to be highly optimized.
- *Both hardware and software aspects have to be considered simultaneously*!

Challenges in embedded system design

- How much hardware do we need?
  - How big is the CPU? Memory?
- How do we meet our deadlines?
  - Faster hardware or cleverer software?
- How do we minimize power?
  - Turn off unnecessary logic? Reduce memory accesses?

Challenges, etc.

- Does it really work?
  - Is the specification correct?
  - Does the implementation meet the spec?
  - How do we test for real-time characteristics?
  - How do we test on real data?
- How do we work on the system?
  - Observability, controllability?
  - What is our development platform?

How much hardware do we need ?

- Selection of µP ,Memory, Peripheral etc..

  ES have to meet

  1. Performance deadlines
  2. Minimum manufacturing cost

- Little hardware – fails to meet deadlines
- More hardware - Makes it expensive

How do we meet deadlines ?

- Speed up the hardware results in faster execution
- Speed may be limited by Memory system

How do we minimize power consumption?

- Excessive power consumption increases heat dissipation

How does the nature of ES make their design more difficult?

- Limited Observation
- Complex Testing
- Limited Development Environment

Embedded processor technology

Book : Frank Vahid

Embedded processor technology

- General-purpose processors -- software
- Single-purpose processors – hardware
- Application-specific processors

Embedded processor technology
General-purpose processors -- software

- The designer of a general-purpose processor builds a device suitable for a variety of applications, to maximize the number of devices sold
- One feature of such a processor is a program memory
- Another feature is a general datapath – the datapath must be general enough to handle a variety of computations, so typically has a large register file and one or more general-purpose arithmetic-logic units (ALUs)

Embedded processor technology
General-purpose processors -- software

- Design time and NRE cost are low, because the designer must only write a program, but need not do any digital design.
- Flexibility is high, because changing functionality requires only changing the program.
- Unit cost may be relatively low in small quantities, since the processor manufacturer sells large quantities to other customers and hence distributes the NRE cost over many units.
- Performance may be fast for computation-intensive applications, if using a fast processor, due to advanced architecture features and leading edge IC technology.

Design-metric drawbacks.

- Unit cost may be too high for large quantities.
- Performance may be slow for certain applications. Size and power may be large due to unnecessary processor hardware.

Embedded processor technology
Single-purpose processors -- hardware

- Single-purpose processors -- hardware A single-purpose processor is a digital circuit designed to execute exactly one program. For example, consider the digital camera
- All of the components other than the microcontroller are single-purpose processors. The JPEG codec, for example, executes a single program that compresses and decompresses video frames. An embedded system designer creates a single-purpose processor

Embedded processor technology
Single-purpose processors -- hardware

- Performance may be fast,
- size and power may be small, and
- unit-cost may be low for large quantities, while design time and NRE costs may be high,
- flexibility is low,
- unit cost may be high for small quantities, and performance may not match general-purpose processors for some applications.

Embedded processor technology
Application-specific processors

- An application-specific instruction-set processor (or ASIP) can serve as a compromise between the above processor options.
- An ASIP is designed for a particular class of applications with common characteristics, such as digital-signal processing, telecommunications, embedded control, etc.
- The designer of such a processor can optimize the datapath for the application class, perhaps adding special functional units for common operations, and eliminating other infrequently used units

Embedded processor technology
Application-specific processors

- Embedded system can provide the benefit of flexibility while still achieving good performance, power and size.
- However, such processors can require large NRE cost to build the processor itself, and to build a compiler, if these items don't already exist

Figure 1.6: Implementing desired functionality on different processor types: (a) general-purpose, (b) application-specific, (c) single-purpose.